

Reinforcement Learning Project Proposals

Rules:

- Students may work individually or in teams of two.
- Students can choose a topic from the provided list or propose their own project.
- Custom project proposals require approval from the lab instructor and must include a concise description.
- By **April 27**, each team must submit their composition and chosen project topic ( RL projects -teams).
- Teams may be formed across different groups. In such cases, both groups and both instructors should be listed in the submission sheet. The instructors will decide individually whether the project will be supervised by one of them or jointly.
- Ambitious students are encouraged to consider developing “publishable” versions of their projects.

Grading

- The project is worth **70%** of the total grade from the labs.
- The general direction of the projects and the expected form of the final evaluated outcomes are outlined in each proposal.
- Projects will also be evaluated based on the achievement of goals agreed upon with the instructor. Students have influence over the direction of their projects.
- In the case of negative results, the evaluation will also consider the rigor of preparation, quality of analysis, and overall approach.
- Students are expected to provide regular (very brief) updates on their progress during lab sessions.
- At the end of the semester, students must present the final results of their project to the rest of the group during class.
- One of the final deliverables should be a short report (3-4 pages), formatted as a mini research paper.
- LLM policy: You may use any AI/LLM/coding tools you wish during the project preparation. Their usage should be briefly documented in a final report. Describe what tools were used, what for, what worked and what didn't work.

Project proposals

Proposal 1: Scaling Contrastive RL with Transformer Architectures

Overview & Motivation

Recent breakthroughs in self-supervised reinforcement learning, highlighted by the NeurIPS 2025 Best Paper, "1000 Layer Networks for Self-Supervised RL", have challenged the conventional wisdom that RL networks collapse when scaled. By replacing the traditional regression-based value critic with a contrastive learning objective (InfoNCE), CRL frames goal-reaching as a classification problem. This stabilizes the training signal, allowing networks to scale up to 1000 layers and unlocking qualitatively new emergent behaviors in pathfinding and goal-reaching.

Given these massive gains achieved using deep Residual MLPs, a natural question arises: can we achieve even better performance, sample efficiency, or representation learning by moving to a Transformer architecture? Transformers have dominated scaling laws in vision and language due to their self-attention mechanisms, but their capabilities in this new ultra-deep self-supervised RL setting remain largely unexplored.

Project Objective

The primary goal of this project is to integrate a Transformer-based architecture into the CRL framework. The project will investigate whether utilizing self-attention mechanisms and sequence-friendly architectures provides tangible performance or scalability benefits over the newly established deep ResNet baselines.

Expected Deliverables (Pass Criteria)

To successfully pass this project, students are expected to complete the following concrete tasks:

- **Baseline Setup:** Set up a working codebase for Contrastive RL based on the recent "1000 Layer Networks" paradigm. Reproduce a baseline run using a deep Residual MLP architecture.
- **Architecture Implementation:** Design and implement a Transformer encoder backbone to replace the MLP in the CRL critic (and optionally the actor). You will need to project the state and goal embeddings into a sequence of tokens that the Transformer blocks can process.
- **Benchmarking & Evaluation:** Train both the baseline deep-MLP CRL and your newly built Transformer-CRL on at least two standard goal-conditioned environments (e.g., robotic manipulation in Fetch environments, or continuous locomotion in AntMaze/Brax).
- **Comparative Analysis:** Deliver a final report comparing the two models. You must control for parameter count (ensuring both networks are roughly the same size) and compare their sample efficiency, final goal-reaching success rate, and training stability.

Advanced Extensions (For students who want to do more)

If you complete the base requirements quickly, you can explore the following advanced directions:

- **Sequence Context (Temporal History):** Instead of feeding a single state-goal pair into the network, leverage the Transformer's true strength by passing in a rolling window of historical states (similar to a Decision Transformer). Evaluate if this temporal context allows the CRL agent to solve Partially Observable MDPs (POMDPs) where standard deep MLPs fail.

- **Depth Scaling & Emergent Behaviors:** Push the Transformer architecture to extreme depths (e.g., 20–50+ attention blocks). Investigate whether Transformers exhibit "sudden jumps" in emergent path-stitching capabilities at critical depths as observed in the original MLP-based paper.
- **Attention Map Interpretability:** Extract and visualize the attention weights from the trained Transformer critic. Analyze what the network is "looking at" during successful trajectories, does the attention mechanism naturally learn to focus on environmental bottlenecks, subgoals, or collision states?
- **Width vs. Depth Trade-off in Attention:** Experiment with the trade-off between the embedding dimension (width) and the number of Transformer blocks (depth) under a strict parameter budget to see which scaling axis benefits self-supervised RL the most.

Proposal 2: Micro-VLA: Scaling In-Context Robotic Manipulation via Observation-Only Videos

Overview & Motivation

Vision-Language-Action (VLA) models, such as RT-2 and OpenVLA, have recently demonstrated incredible zero-shot generalization in robotics. However, these models face a severe data bottleneck: while we have massive repositories of video data showing humans or robots completing tasks, we have very little *action-annotated* data containing the exact motor torques and joint commands required to replicate those movements.

Standard supervised approaches fail when actions are missing. If we can design architectures that learn world dynamics and physics directly from "observation-only" videos, we can vastly expand the training sets for robotic foundation models. This project explores whether pre-training on a large corpus of action-free manipulation videos can improve a small-scale VLA's ability to perform In-Context Learning (ICL) from just one or two fully annotated demonstrations.

Project Objective

The primary goal of this project is to build a "Micro-VLA" agent that improves its sample efficiency and few-shot generalization by learning from unannotated video trajectories. The student will implement a system that uses an Inverse Dynamics Model (or masked token prediction) to ingest observation-only sequences from a simulated robotics environment, combining this with a small set of action-annotated data to solve continuous control manipulation tasks.

Expected Deliverables (Pass Criteria)

To successfully pass this project, students are expected to complete the following concrete tasks:

- **Simulated Benchmark Setup:** Set up a lightweight, continuous-control robotic manipulation environment (e.g., Robomimic or Meta-World). Generate a synthetic dataset: a small fraction containing full state-action-reward data, and a large fraction stripped of actions to simulate "video-only" observations.
- **Architecture Implementation:** Design a small Transformer-based policy (e.g., a miniaturized Decision Transformer). Implement an auxiliary objective, such as an Inverse Dynamics module, that forces the network to predict the missing actions between two consecutive video frames, allowing it to learn from the observation-only dataset.
- **Benchmarking & Evaluation:** Train two models: a baseline Micro-VLA trained strictly on the small, fully-annotated dataset, and your proposed Micro-VLA trained on the mixed dataset.

- **Comparative Analysis:** Deliver a final report evaluating the models on their In-Context Learning capabilities. Specifically, prompt the frozen models with a single successful demonstration of a new, unseen task variation (e.g., picking up a differently colored object) and measure which model exhibits better zero-shot or few-shot transfer.

Advanced Extensions (For students who want to do more)

If you complete the base requirements quickly, you can explore the following advanced directions:

- **Cross-Embodiment Generalization:** Collect your observation-only video data using one type of robot arm (e.g., a Sawyer arm), but evaluate the In-Context Learning agent on a completely different kinematic structure (e.g., a Franka Emika Panda arm). Investigate whether learning general physics from the first robot transfers to the second.
- **Action Chunking for Error Mitigation:** Inverse dynamics models are prone to compounding errors. Implement an "Action Chunking" mechanism (similar to the ACT architecture) where the Transformer predicts a sequence of future actions rather than just the next step. Evaluate if this temporal smoothing improves the stability of the rollout.
- **Learning from Sub-optimal Videos:** The real world is full of mistakes. Introduce a large percentage of failed trajectories into your observation-only dataset. Modify your architecture to condition on a "success/failure" token or return-to-go. Can the model learn useful dynamics from watching failures without copying the bad behavior?
- **Test-Time Adaptation:** While In-Context Learning is weight-free, explore whether taking the prompt demonstration and performing just 5–10 gradient update steps (online fine-tuning) on the frozen model's final layers dramatically boosts the success rate compared to pure ICL.

Inspiration:

- <https://arxiv.org/abs/2312.03801>

Proposal 3: Scaling Compute and Sample Efficiency in PPO

Overview & Motivation

Proximal Policy Optimization is a workhorse in modern RL, but standard implementations usually rely on simple MLPs. With hardware-accelerated simulators (like JAX environments), we can collect massive amounts of data in minutes. In theory, scaling up the network capacity and the number of parallel environments should lead to huge performance gains.

In practice, however, PPO often hits a wall. Recent work shows that upgrading architectures (e.g., adding Residual Blocks and Layer Normalization) helps absorb this data better, but even then, scaling eventually saturates. Because all parallel environments are driven by a single policy, the collected data becomes highly homogeneous. This lack of data diversity limits the benefits of scaling, presenting a fundamental bottleneck.

Project Objective

The main goal of this project is to scale PPO's network capacity using modern architectural tweaks and systematically push it until it breaks. You will investigate *when* and *why* scaling stops working, specifically analyzing how homogeneous data from a single policy causes data saturation and gradient variance, and lay the groundwork for mitigating these issues.

Expected Deliverables (Pass Criteria)

To successfully pass this project, students are expected to complete the following concrete tasks:

- **Baseline Setup:** Set up a working, highly parallelized PPO codebase natively in JAX (e.g., leveraging principles from PureJaxRL) to ensure end-to-end compilation (both the environment and training loop on the GPU/TPU). Establish baseline learning curves highlighting the point of diminishing returns when scaling the number of parallel environments.
- **Architecture Implementation:** Upgrade the PPO actor and critic networks by integrating Residual Blocks and Layer Normalization. Build the network so that depth and width can be easily scaled up via configuration.
- **Benchmarking & Evaluation:** Train both the baseline MLP-PPO and your enhanced ResBlock-PPO across JAX-native RL domains. To ensure fast iteration, you must evaluate the models on at least one continuous control suite (Brax) and one complex, open-ended discrete environment (Craftax). Standard environments from Gymnax can be used for rapid initial prototyping.
- **Comparative Analysis:** Deliver a report analyzing *why* the scaling breaks down at that threshold. You must track and plot:
 - **Gradient Variance:** How the variance of the policy gradients behaves as the environment count scales up.
 - **Data Saturation:** A simple metric showing the lack of diversity in the rollout batches (e.g., dropping entropy or state distribution overlap) caused by the single-policy data collection.

Proposal 4: Contrastive RL latent mining

Overview & Motivation

CRL uses a contrastive approach to training Critic in a classical Actor-Critic RL setup. This eliminates the need for external rewards and produces state-action and goal encoders that can be used in tandem as a goal-conditioned Q-function. Encoders produce a latent space that encodes the information about distances and relationship between states of the environment. Depending on parametrization, distance function and loss function used this latent space can have interesting properties. In particular it should encode information about the dynamics and properties of the environment. The question is: How much can we learn about the specifics of the environment that the model was trained in, just by examining the latent space? And can knowledge about the shape of the latent space be later used for more advanced reasoning?

Project Objective

The main goal of this project is to understand: What exactly is encoded in the latent space of CRL? In particular can the encoded information be extracted in a way that is understandable to a human? Can it be useful for the algorithm?

Expected Deliverables (Pass Criteria)

To successfully pass this project, students are expected to complete the following concrete tasks:

- **Setup:**
 - Set up JaxGCRL, with experiment logging, and do training sample runs on Simple maze and Ant maze. The first of the environments should be used for debugging and testing, it is very easy and there are no complexities related to movement dynamics. The second one is more interesting, the latent space has to encode not only the

location information, but also more intricate information related to dynamics of the Ant model.

- Prepare a couple (≥ 4) of mazes to test the latent representations and train models on them. Think about their shapes in terms of the rest of the project. You should design them to test specific things.
- Train CRL on those mazes. Novel findings suggest that increasing the network depth in CRL produces higher-quality representations of the environment. The trained networks should have *some* depth.
- Use the trained checkpoints to produce rollouts.
- **Exploration of the latent space:**
 - Visualize rollouts and their associated latent spaces.
 - Visualize the shape of the latent space using PCA/t-SNE/UMAP (or something else) with random points spread throughout the maze. Can the structure of the maze be seen on visualizations? (NOTE: this is an active research area, although we expect the structure of the maze to be visible in some way, it is also possible that the result will be negative - **And that is OK ;)**)
 - Assess how changing the structure of the maze changes the visualizations.
- **Exploiting the latent space:**
 - Can we train a latent space decoder? When given a frozen CRL model, can we train (in a supervised manner) a decoder that takes the latent space of an encoded state, and outputs back this state (the architecture here is somewhat similar to AutoEncoder, but the encoder is frozen).
 - Take a state and goal in the maze. Encode them and interpolate in the latent space between them. Then decode each of the interpolated latent space points. Do they form a path in the maze?
 - What happens if we condition the algorithm on the decoded, interpolated states? Are they meaningful waypoints?

Advanced Extensions (For students who want to do more)

If you complete the base requirements quickly, you can explore the following advanced directions:

- Heuristic bottleneck identification: if the maze has a bottleneck, can it be identified in the latent space? Likely this needs some heuristic algorithm that analyzes the clustering of the latent points.
 - Bottlenecks as waypoints: If we can identify the bottlenecks in the maze structure, is using them as subgoals beneficial to the performance?
- Heuristic maze shape reconstruction: given the latent space of an algorithm trained on some maze, can we reconstruct the maze it was trained on? This likely also can be done through heuristic methods and smart clustering.

Resources:

- [Contrastive Reinforcement Learning](#) - the original idea for goal-conditioned unsupervised RL.
- [JaxGCRL](#) - Benchmark and codebase with most of the important things implemented.
- [Why is CRL latent space "nice"](#) - More of a theoretical paper exploring properties of latent space in CRL.
- [Scaling CRL](#) - Recent paper about scaling CRL, among other things, authors show that some network depth increase improves the latent space properties.

Proposal 5: Offline model based RL on OGBench

Overview & Motivation

OGBench is a recent benchmark/dataset collection that concerns offline Goal-Conditioned RL. It provides a number of environments and datasets for each of those environments. It has been used for model-free GCRL mostly for now. However, the environments and dataset present there seem like an ideal place to test world-model learning capabilities.

Project Objective

The main objective is to test if there is a benefit to adding a world model in offline RL in OGBench. We want to train a world model on a given dataset, and then use this world model to train a standard RL algorithm. Once the world model is trained, it is frozen and never changed again. The RL algorithm doesn't have contact with the environment itself, only the world model during training. We are interested, whether adding the world model will bring any benefit, when compared to training offline RL algorithms on the dataset.

Expected Deliverables (Pass Criteria)

To successfully pass this project, students are expected to complete the following concrete tasks:

- Set up OGBench and download datasets for desired environments (at least 2, one manipulation and one locomotion).
- For each environment:
 - Train the world model on a dataset in a supervised manner. (We want the model that given state and action predicts next state, reward and termination).
 - Train at least 2 RL methods implemented in OGBench on the world model. This will require modifying the training loop, to allow for semi-online data gathering. In each epoch we should "hallucinate" trajectories using the world model and current policy. Then those trajectories should be used to train the RL model.
 - Train the same RL methods directly on the dataset.
 - Compare the performance of the trained methods in the real environment. We want to see if they perform similarly, when given the same amount of data, and if the world model method can perform better, when given more rollouts.
- Ablate over the dataset size - Once the entire pipeline above is working, try training it (both model based and model free) on half of the dataset, does it still work? What about 10% of the dataset? Plot the percentage of dataset used vs averaged performance.

Advanced Extensions (For students who want to do more)

If you complete the base requirements quickly, you can explore the following advanced directions:

- State based vs image based: OGBench includes both modalities, which one is better for world model training? Compare both modalities in the same environment. What modifications are necessary? (This may require a bit more compute)

Resources:

- [OGBench](#)

Proposal 6: Reinforcement Learning for Graph Optimization in Causal Discovery

Causal discovery aims to recover the underlying graph structure that encodes relationships between variables in a tabular dataset. A prominent class of approaches formulates this as an optimization problem: given a candidate graph, a scoring function quantifies how well it explains the observed data. This score can naturally be interpreted as a reward signal, enabling the use of reinforcement learning (RL) to search over the space of graphs.

This project builds upon the work of Sypniewski & Olko [1], where graph structures are parameterized using two matrices. Likelihoods—and consequently rewards—are computed using Prior-data Fitted Networks (PFNs), and posterior optimization is performed using Proximal Policy Optimization (PPO). For details, see their paper.

Algorithm 1 Graph Posterior Optimization with PPO

Require: Dataset D , regularization parameter λ , number of iterations T , PPO update frequency K

```

1: Initialize posterior parameters  $(P, W)$ 
2: Split  $D$  into  $D_{\text{train}}$  and  $D_{\text{est}}$ 
3: for  $t = 1, \dots, T$  do
4:   Data Acquisition:
5:   for  $k = 1, \dots, K$  do
6:     Sample adjacency matrix  $A_{t,k}$  from posterior  $(P, W)$  using Gumbel–Softmax and Soft-
       Sort
7:     Compute log-likelihood of sampled adjacency:  $\log \pi_{(P,W)}(A_{t,k})$ 
8:     for each variable  $i \in V$  do
9:       Fit PFN model on  $(D_{\text{train}}, \text{pa}_{A_{t,k}}(i))$ 
10:      Compute likelihood  $\log p_i(D_{\text{est}} \mid D_{\text{train}}, \text{pa}_{A_{t,k}}(i))$ 
11:    end for
12:    Compute reward (score):

$$\hat{s}(A_{t,k}, D) = \sum_{i \in V} \log p_i(D_{\text{est}} \mid D_{\text{train}}, \text{pa}_{A_{t,k}}(i)) - \lambda |A_{t,k}|$$

13:    Store  $(A_{t,k}, \log \pi_{(P,W)}(A_{t,k}), \hat{s}(A_{t,k}, D))$  in trajectory buffer
14:  end for
15:  Policy Update:
16:  Use PPO to update posterior parameters  $(P, W)$  based on stored tuples
     $(A_{t,k}, \log \pi_{(P,W)}(A_{t,k}), \hat{s}(A_{t,k}, D))$ 
17: end for
18: return Learned posterior over graphs

```

There are three possible trajectories for this project:

1. Improving graph parameterization

The goal is to compare existing graph parametrizations, including those which do not guarantee acyclicity of the sampled graph. We could still apply them, but then the RL objective would need to account for cycles. Concretely, we would add a cyclicity penalty to the reward/value and optimize an augmented Lagrangian version of the original score.

2. Reinforcement Learning Algorithmic Improvements

Investigate alternative RL algorithms better suited to this optimization problem. In particular, explore the use of Soft Actor-Critic (SAC) and compare it against PPO in terms of: Sample efficiency, Stability of training, Quality of discovered graph structures.

Alternatively, use existing literature on asynchronous policy gradient methods and formulate improvements to PPO that increase sample efficiency.

The objective is to identify optimal hyperparameter configurations and provide principled explanations for observed performance differences, grounded in the properties of the algorithms and rewards in this problem.

3. Problem specific RL improvements

Optimization can be improved (or reformulated) using the assumption that the environment consists of only one state. We could also leverage the invariances of the score etc.

The objective is to come up with principled methods and compare them.

Expected deliverables

- Setup the environment and pick datasets (graphs)
- In the trajectory of your choosing, show improvements over the baseline, or show that the baseline is optimal.
- Provide a qualitative and quantitative analysis of various approaches

This is an ongoing work and may result in publication at an A* conference.

[1] <https://arxiv.org/abs/2512.11840>

Proposal 7: Optimizing Chain-of-Thought Length in Small Language Models via Reinforcement Learning

Overview

This project investigates how the length of chain-of-thought (CoT) reasoning affects task performance in small language models. Students will first identify the empirically optimal reasoning length through controlled experiments or literature review, then apply reinforcement learning with explicit length regularization to shape model behavior. Finally, they will explore self-distillation as a mechanism for compressing reasoning while preserving accuracy, comparing performance across different target output lengths.

Phase 1: Optimal CoT Length Empirical Analysis – baseline

Students will either:

- Conduct a literature review on CoT length vs performance (focus on recent RL and reasoning-efficiency papers),

or

- Run experiments:

Use a fixed model (e.g., Qwen-0.5B)

Pick a method of shortening CoT of their choosing: E.g. final answer elicitation, Prompting, Decoding (beam search with length penalty?)

Tasks:

arithmetic reasoning (e.g., MATH, Ace, Polaris, omni math) or
simple logical tasks (e. g. ARC AGI)

Phase 2 (Main): RL with Length Regularization

Model: small Qwen (0.5B–1.5B)

RL method: PPO or GRPO

Dataset: small dataset of reasoning problems

Reward: $R = R_{\text{correctness}} - \lambda \cdot \text{length}$

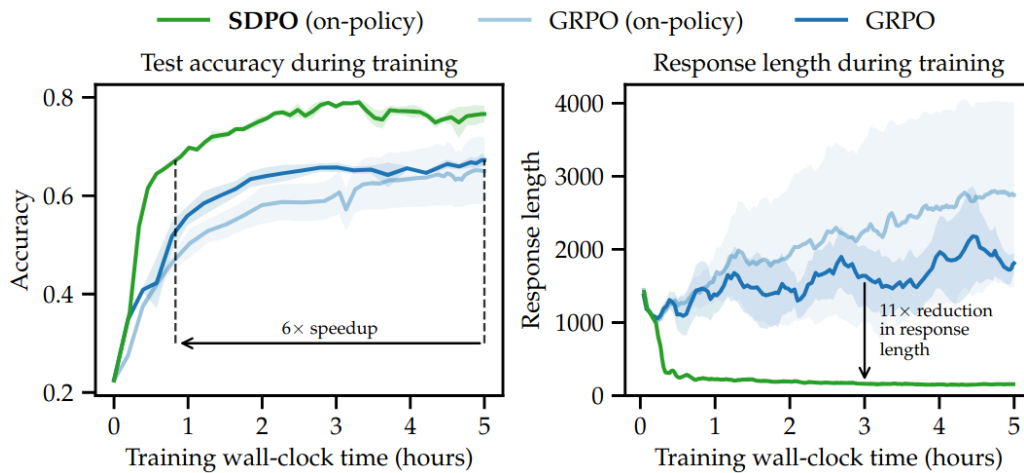
Experiments: Vary λ (e.g., 0.0, 0.01, 0.05, 0.1)

Measure: accuracy, average length, etc.

Goal: trade-off between brevity and correctness

Phase 3 (optional): Self-Distillation for Length Control

See if reinforcement learning via self-distillation can be better used to control the length of reasoning traces of a student. <https://github.com/lasgroup/SDPO> .



[1] <https://arxiv.org/abs/2502.07266>

[2] <https://arxiv.org/abs/2601.20802>

Proposal 8: Epsilon-greedy to Boltzmann

The epsilon-greedy exploration strategy is commonly used for discrete control tasks. An alternative is the Boltzmann strategy. On the first stage of the training, it makes a lot of sense to use the epsilon-greedy strategy as our estimator of the Q-function is noisy and thus little informative. However, in further stages of the training, it makes more sense to gradually switch to the Boltzmann strategy, as it inherits the prior from the reasonable Q-function. The project aims to study how to gradually transfer the exploration strategy from epsilon-greedy to the Boltzmann strategy. The first idea is a simple annealing from the epsilon-greedy strategy to the Boltzmann one, with gradually changing the temperature.

Another approach is to choose a number of actions (e.g., 2-3), and instead of choosing the greedy action, sample from the Boltzmann distribution on these actions. Such an approach should allow for exploring more actions that are good according to the Q-function, but are not best (possible due to the Q-function inaccuracies).

The first iteration of the project should be tested on OpenAI Gym environments (for example, Lunar Lander). These are toy environments, but they are fast to evaluate, and one can find bugs/implementation issues. The next step can be an evaluation on the Atari benchmark. The base algorithm for the implementation should be some variant of the DQN, with extensions towards Rainbow.

The minimal requirement is to implement a few variants of the aforementioned ideas and to test them on OpenAI Gym environments.

Extending the project includes an evaluation on the Atari benchmark and more careful crafting of the switching between the epsilon-greedy and Boltzmann exploration, for example, based on the uncertainty of the Q-function (when the uncertainty of the Q-function is low, we should direct more towards the Boltzmann exploration, when the uncertainty is high, the epsilon-greedy strategy is more reasonable).

Proposal 9: Correlated exploration for MARL

In multi-agent reinforcement learning, exploration is usually performed by applying the epsilon-greedy strategy independently for each agent. However, if agents are correlated, it would be reasonable to correlate their exploration to try new collective behaviours. The project aims to find a strategy for correlated exploration between the agents.

We hypothesize that agents that are in similar states should explore similar actions to find more consistent strategies. The project aims to compute the correlation between agents based on some quantity of interest (for example, observations, hidden representations of observations' history, Q-functions). Then we want to create a correlated distribution for exploration based on this correlation measure. Agents will explore using the epsilon-greedy strategy, but not independently, leading to coordinated exploration.

To transfer correlation between agents to a correlated exploration distribution, one can use the Gaussian copula method ([https://en.wikipedia.org/wiki/Copula_\(statistics\)](https://en.wikipedia.org/wiki/Copula_(statistics))). This method requires providing a correlation matrix that is positive semidefinite. Thus, the correlation measure must satisfy such an assumption. This can be done, for example, by considering cosine similarity (which creates a Gram matrix, which is positive semidefinite).

As a base algorithm, a value-decomposition algorithm (such as VDN, QMIX, QPLEX, NA²Q) should be used. First evaluation should be performed on toy environments, such as Level-Based Foraging, Hallway. In the second stage, experiments could be conducted on StarCraft-based environments (SMAC, SMACv2, SMAX) and Google Research Football.

Proposal 10: Target entropy adjustment for SAC

In SAC, the target entropy is usually given by some heuristic connecting it to the action dimension. However, when the agent's policy is well-trained, it might be useful to use a lower entropy coefficient than at the beginning, when the policy should explore more. The project aims to find a strategy of adjusting the target entropy for SAC based on the uncertainty of the agent's policy.

The experiments can be conducted in the first stage on the Pendulum as a toy environment and the MuJoCo benchmarks. Then, they can be transferred to the DMC Suite and MyoSuite.

Proposal X: The "Policy" is just Python Code

Overview & Motivation

Recent work on *Trace* proposes an AutoDiff-like framework for end-to-end generative optimization of AI systems that receive rich, non-differentiable feedback. Instead of optimizing only numeric weights, *Trace* treats heterogeneous objects such as prompts and code as optimizable parameters, and uses execution traces together with feedback on the output to update them through a Python library with PyTorch-like syntax. The authors position it as a general platform for optimizing agentic workflows, and report applications spanning prompt optimization, code debugging, and robot controller design.

This idea is especially interesting in the context of reinforcement learning and agentic systems such as *Voyager*, where skills are stored as executable code and improved through an iterative loop driven by environment feedback, execution errors, and self-verification. At the same time, code-as-policy methods are often difficult to scale: as tasks require longer horizons, more reusable skills, and more exception handling, the self-improvement loop itself becomes cumbersome to engineer. One useful way to think about this is that many current systems still “implement backprop by hand” at the workflow level. A natural question arises: can *Trace* provide a cleaner and more principled way to optimize code-based policies for sequential decision-making, especially on tasks that require composing multiple skills?

Project Objective

The primary goal of this project is to build a *Trace*-based code-as-policy agent and investigate whether generative optimization can solve reinforcement learning tasks of increasing compositional difficulty. In particular, the project will study whether *Trace* can simplify the implementation of a self-improvement loop, and whether it enables better skill acquisition, reuse, and composition than a fixed code policy or a manually engineered reflection/improvement baseline.

Expected Deliverables (Pass Criteria)

To successfully pass this project, students are expected to complete the following concrete tasks:

- **Baseline Setup:** Set up a lightweight, grid-based or discrete control environment that allows for progressive task complexity and skill composition (e.g., Crafter, BabyAI, or a simplified Minecraft-like gridworld). Implement a basic "Code-as-Policies" LLM agent without automated self-correction as a baseline.
- **Trace Integration:** Implement an optimization workflow using the *Trace* library. Formulate the agent's prompts and the generated Python/execution code as the system's "parameters," and use the environment's reward signal and execution errors as the feedback mechanism for the optimizer.
- **Skill Scaling & Benchmarking:** Train and evaluate the *Trace*-optimized agent on a curriculum of tasks. Begin with simple, single-skill objectives (e.g., "navigate to an object") and scale up to complex objectives requiring multiple composite skills (e.g., "gather resources, craft a tool, and defeat an enemy").
- **Comparative Analysis:** Deliver a final report evaluating the framework's effectiveness. You must specifically analyze the agent's robustness against edge cases as complexity scales. Discuss the qualitative differences between *Trace*'s feedback-driven "predictive coding" optimization and standard gradient-based RL, highlighting what worked and where the abstraction broke down.

Advanced Extensions (For students who want to do more) If you complete the base requirements quickly, you can explore the following advanced directions:

- *Trace* relies heavily on feedback to optimize its "parameters." But what happens when the environment only provides a sparse, binary "success/fail" signal at the very end of a long

rollout? How can you design a system where the agent autonomously generates dense, intermediate feedback (e.g., self-critiques or heuristic sub-goals) to guide the Trace optimizer when the environment gives it nothing?

- When the optimizer refactors a Python policy to handle a newly discovered edge case, it risks breaking the logic that worked for previously solved scenarios. How can you prevent this? Can you design a novel mechanism, such as automated regression testing, experience replay, or modular skill-freezing, that forces the optimizer to preserve past capabilities while adapting to new ones?
- Code-as-policies approaches inherently assume predictable rules. If the environment dynamics suddenly shift (e.g., an object's physics change or a new adversarial obstacle appears), the static code will fail. How can the framework quickly detect this distribution shift and trigger a targeted, sample-efficient re-optimization of only the affected logic, rather than rewriting the entire policy from scratch?

Resources

- [trace paper](#), [metaworld example](#)
- [voyager paper](#)
- [code as policies paper](#)
- alternative to trace - [textgrad](#)